DESAFIO DE INOVAÇÃO DATA SYSTEM 2026

Onde usamos IA

FATECalçados · Programa de Fidelidade

> **Ferramenta:** Claude (Anthropic), usado como assistente de programação dentro do editor.
> **Como trabalhamos:** a equipe definia o que fazer e as regras do negócio; a IA propunha e escrevia o código; a equipe revisava, testava na tela e pedia ajustes. Nos trechos mais complexos, o código tem um comentário `Feito com IA:` explicando o que ele faz.

Onde, por quê e o que validamos

Onde	Por quê	O que validamos
Regras do programa e backend (Django) `regras.py`, `services.py`, `models.py`	Traduzir o regulamento em código com rapidez e com um teste para cada regra.	Conferimos cada regra com a página 12 do desafio. Revisamos as decisões que o PDF deixa em aberto (nível pelos pontos acumulados, bônus pelo nível de antes da compra, pontos sobre o valor pago). 50 testes automáticos passando.
Dados de seed `popular_dados`	Precisávamos de dados coerentes com as regras. A IA sugeriu gerar *vendas* e passá-las pelas mesmas regras do caixa, em vez de inventar pontos.	40 clientes e 420 compras com CPF (mínimo do desafio: 20 e 200). Conferimos no dashboard que custo = pontos resgatados × R$ 0,05. Testamos os CPFs do relatório no caixa.
Organização do frontend (React)	O protótipo de telas era um arquivo único, com dados fictícios. Queríamos código simples de ler e explicar.	Revisamos o código e pedimos para simplificar trechos complicados (por exemplo, o gráfico feito à mão virou a biblioteca Recharts). Removemos os dados fictícios.
Integração do caixa (PDV) com a API	Ligar a tela ao backend com segurança: preço vindo do banco, venda offline sem duplicar, resgate de pontos.	Testes da API e testes clicando na tela: venda com resgate, venda offline e sincronização. O recibo mostra os mesmos pontos que a regra calcula.
Dashboard gerencial	Montar os blocos pedidos no Entregável 2.	Comparamos bloco a bloco com a página 11 do desafio e usamos os mesmos termos na tela. Testamos o filtro de período.
Aba Ações	Responder à dor da diretoria ("eles compram e somem"). A IA sugeriu as quatro listas: sumindo, pontos vencendo, perto de subir de nível, sem segunda compra.	Conferimos as listas com os dados do banco. Cada lista tem um teste que confere quem entra e quem não entra.
Revisão do código	Encontrar erros antes da apresentação.	A revisão encontrou três problemas, todos corrigidos com teste: pontos que podiam expirar duas vezes em consultas simultâneas, contagem de compras incluindo vendas sem CPF e venda com data no futuro.
Documentos e relatórios	Explicar as regras para a equipe e ter listas de clientes e produtos para testar.	Os exemplos do documento de pontos foram conferidos na própria função do backend.

O que foi decisão da equipe

- Seguir o regulamento do desafio em vez das regras do protótipo (que dava desconto por nível e bônus no Pix).
- Remover o que não estava alinhado com o desafio: sangria, antifraude, desconto do gerente e matriz RFM.
- Criar a aba Ações para mostrar ao gestor *o que fazer*, e não só números.

- Usar na tela os mesmos termos do desafio (Base do programa, Custo do programa, Evolução no tempo, Ranking).
- Refazer o quadro de pontos do caixa depois de acharmos a primeira versão confusa.

Onde a IA errou e corrigimos

- Uma mudança no seed trocou todos os clientes sem aviso claro: os CPFs do nosso relatório pararam de funcionar no caixa. Geramos o relatório de novo.
- A primeira versão do gráfico calculava o desenho à mão, código difícil de explicar. Trocamos por uma biblioteca.
- O quadro de pontos misturava "pontos ganhos" e "pontos usados" num lugar só. Reorganizamos em: saldo → usar → ganhar → saldo depois.
- No celular, os gráficos passavam da borda da tela. Percebemos no teste e corrigimos.

Como garantimos que entendemos o projeto

- **Testes automáticos** para cada regra do programa: se uma regra quebrar, um teste avisa.
- **Documento de pontos, bônus e desconto** (`docs/pontos-e-bonus.pdf`), com exemplos e onde cada regra está no código.
- **Comentários** `Feito com IA:` nos trechos mais complexos, explicando o que fazem.
- **README do backend** com um roteiro de estudo para a equipe antes da banca.